

# Probabilistic Machine Learning

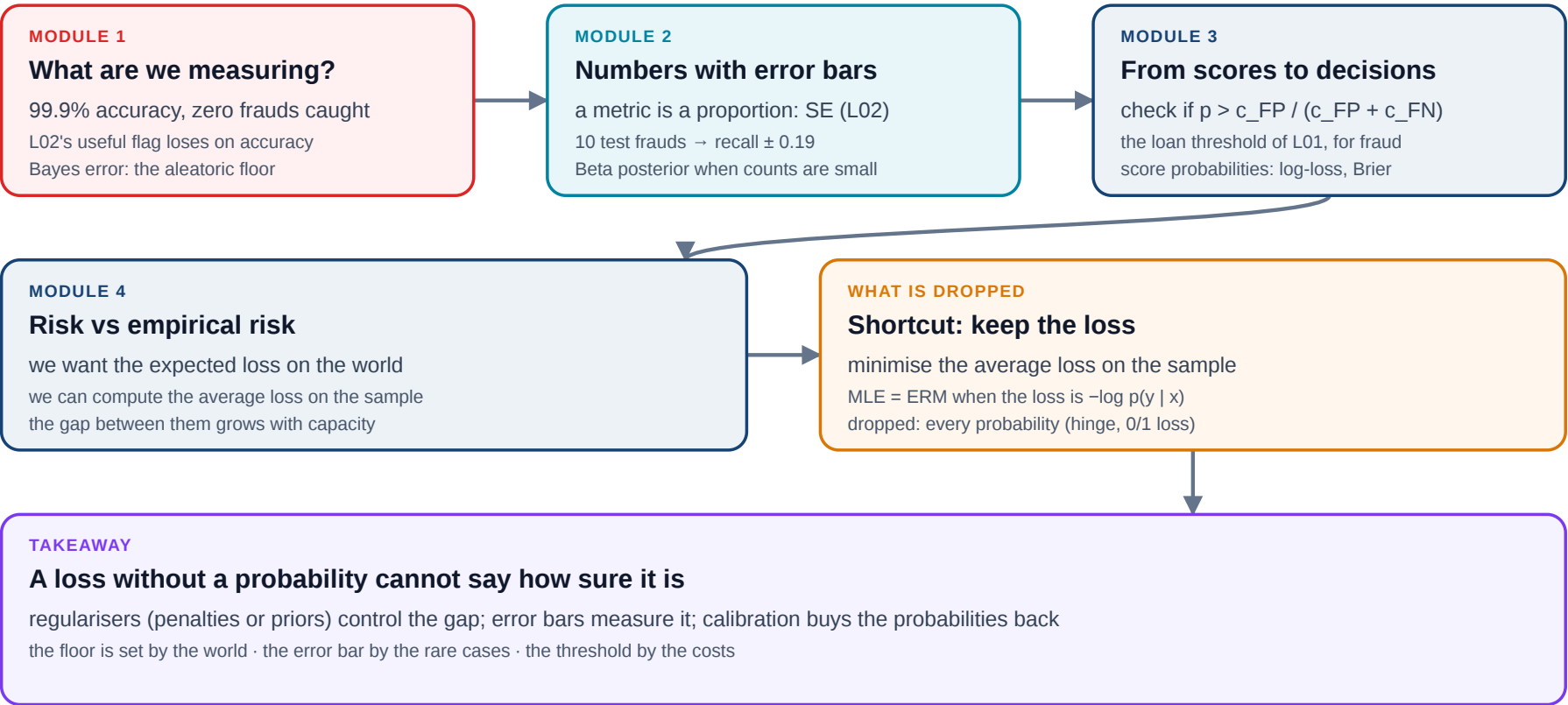
---

## Lecture 06: Evaluation and Empirical Risk Minimisation

Prof. Dr. Michael Granitzer

# Argument Map

## Lecture 06 · argument map



Course arc  
ideal → shortcuts

L01  
The ideal

L02  
The language

L03  
Peak · apart

L04  
Only  $p(y | x)$

L05  
Graph the joint

L06  
Keep the loss

L07  
No shortcut

# Module 1

---

## What Are We Measuring?

**Guiding Question:** A fraud model reports 99.9% accuracy and has never caught a single fraud. What should we have measured instead — and how low could the error of the best model be?

# What We Want to Know

The **true error** of a classifier  $h$  is the probability that it errs on a fresh example drawn from the world:

$$\text{err}^*(h) = P(h(X) \neq Y), \quad (X, Y) \sim p$$

The **Bayes error** is the smallest true error any classifier can reach:

$$\min_h \text{err}^*(h) = \mathbb{E}_X \left[ \min(p(y=1 \mid X), 1 - p(y=1 \mid X)) \right]$$

A transaction type is fraudulent 55% of the time. Could a better model push its error below 45%?

# What We Want to Know

The **true error** of a classifier  $h$  is the probability that it errs on a fresh example drawn from the world:

$$\text{err}^*(h) = P(h(X) \neq Y), \quad (X, Y) \sim p$$

The **Bayes error** is the smallest true error any classifier can reach:

$$\min_h \text{err}^*(h) = \mathbb{E}_X \left[ \min(p(y=1 \mid X), 1 - p(y=1 \mid X)) \right]$$

A transaction type is fraudulent 55% of the time. Could a better model push its error below 45%?

No — only **new attributes** can. The Bayes error is aleatoric uncertainty (Lecture 01's identical patients) written as an error rate: the floor that no amount of data removes.

# Lecture 02's Flag, Evaluated

| per 100,000 transactions, 100 frauds         | always "legitimate" | Lecture 02's flag |
|----------------------------------------------|---------------------|-------------------|
| frauds caught · honest customers flagged     | 0 · 0               | 99 · 999          |
| <b>accuracy</b>                              | <b>99.9%</b>        | <b>99.0%</b>      |
| recall $P(\text{flag} \mid \text{fraud})$    | 0%                  | 99%               |
| precision $P(\text{fraud} \mid \text{flag})$ | —                   | 9%                |

Which model would you deploy — and which one does accuracy recommend?

# Lecture 02's Flag, Evaluated

| per 100,000 transactions, 100 frauds         | always "legitimate" | Lecture 02's flag |
|----------------------------------------------|---------------------|-------------------|
| frauds caught · honest customers flagged     | 0 · 0               | 99 · 999          |
| <b>accuracy</b>                              | <b>99.9%</b>        | <b>99.0%</b>      |
| recall $P(\text{flag} \mid \text{fraud})$    | 0%                  | 99%               |
| precision $P(\text{fraud} \mid \text{flag})$ | —                   | 9%                |

Which model would you deploy — and which one does accuracy recommend?

By accuracy, the model that catches **nothing** wins. On a rare class, accuracy counts the majority and hides that a model is useless.

# What to Report Instead

- **A baseline** next to every number: always-majority for classification, the mean for regression. "99.9%" is not a result until you know that doing nothing scores 99.9%.
- **Measures that condition on the rare class**: recall conditions on the truth, precision on the decision — the two directions of Bayes' theorem that Lecture 02 warned not to swap.
- **The costs**: which of the two mistakes the application pays for. Module 3 turns them into a threshold.

Precision depends on the base rate, recall does not: the same flag has precision 9% at 1 in 1,000 and about 50% at 1 in 100.

# Mini-Exercise

A model flags **every** transaction as fraud (one in 1,000 is fraudulent).

What are its recall and its precision?

# Mini-Exercise

A model flags **every** transaction as fraud (one in 1,000 is fraudulent).

What are its recall and its precision?

Recall **100%** — it catches every fraud. Precision **0.1%** — the base rate. (Accuracy: 0.1%.)

Recall alone can be bought by flagging everything; that is why it is always reported together with precision, or with the costs.

# Takeaway

We want the **true error**, whose floor — the **Bayes error** — is aleatoric uncertainty that no data removes. On a rare class, accuracy measures the majority: report a **baseline** and measures that condition on the rare class.

# Module 2

---

## Numbers With Error Bars

**Guiding Question:** The test set says recall 0.90. Would you promise the bank that the model catches 90% of next month's frauds?

# Where an Error Number Comes From

| estimate                                    | how                                                    | what it is                                                                                   |
|---------------------------------------------|--------------------------------------------------------|----------------------------------------------------------------------------------------------|
| <b>training error</b>                       | evaluate on the data the model was fitted on           | optimistically biased — on average below the true error of the fitted model, often far below |
| <b>hold-out</b>                             | split once; fit on one part, evaluate on the other     | an unbiased estimate of $\text{err}^*$ — for <b>one</b> split, with a sampling error         |
| <b><math>k</math>-fold cross-validation</b> | each of $k$ folds serves once as the test set; average | uses every example for testing once; still an estimate from a sample                         |

A test set that is consulted for choices — which model, which threshold — becomes training data. Keep one set that is looked at once.

# A Metric Is a Proportion

Accuracy on  $n$  test cases is the mean of  $n$  yes/no indicators — a Bernoulli sample (Lecture 02). Its standard error is  $\sigma/\sqrt{n}$  with  $\sigma^2 = p(1 - p)$ :

$$\text{SE}(\hat{p}) = \sqrt{\frac{\hat{p}(1 - \hat{p})}{n}}$$

A test set of **10,000** transactions, one in 1,000 fraudulent:

**0.990 ± 0.002**

accuracy:  $n = 10,000$ ,  $\text{SE} \approx 0.001$

**0.90 ± 0.19**

recall, 9 of 10 frauds caught:  $n = 10$ ,  $\text{SE} \approx 0.095$

Same test set, same model — why is one error bar a hundred times wider?

# A Metric Is a Proportion

Accuracy on  $n$  test cases is the mean of  $n$  yes/no indicators — a Bernoulli sample (Lecture 02). Its standard error is  $\sigma/\sqrt{n}$  with  $\sigma^2 = p(1 - p)$ :

$$\text{SE}(\hat{p}) = \sqrt{\frac{\hat{p}(1 - \hat{p})}{n}}$$

A test set of **10,000** transactions, one in 1,000 fraudulent:

**0.990 ± 0.002**

accuracy:  $n = 10,000$ ,  $\text{SE} \approx 0.001$

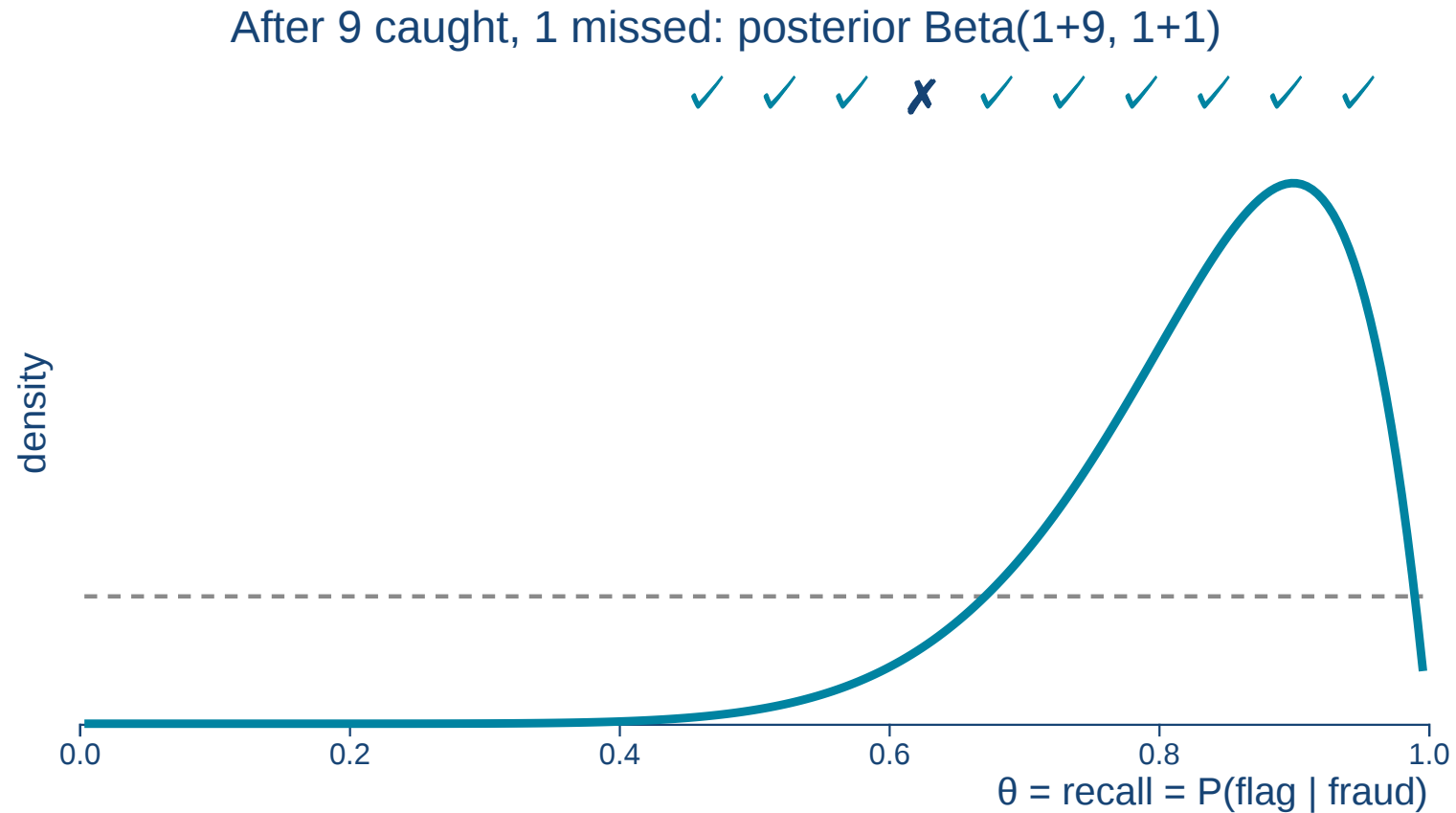
**0.90 ± 0.19**

recall, 9 of 10 frauds caught:  $n = 10$ ,  $\text{SE} \approx 0.095$

Same test set, same model — why is one error bar a hundred times wider?

Recall is a proportion **of the frauds**, and there are only ten of them. The error bar is set by the rare class, not by the size of the test set.

# Recall as an Unknown Parameter



After 9 caught and 1 missed, with a flat prior:  $\text{Beta}(10, 2)$  — mean **0.83**, 95% credible interval **[0.59, 0.98]**.

# The Rare Class Sets the Test Size

Recall near 0.9, so  $SE = \sqrt{0.09/n_{\text{fraud}}}$ ; at one fraud in 1,000 transactions:

| frauds in the test set | transactions | SE of recall | error bar ( $\pm 2\text{ SE}$ ) |
|------------------------|--------------|--------------|---------------------------------|
| 10                     | 10,000       | 0.095        | $\pm 0.19$                      |
| 100                    | 100,000      | 0.030        | $\pm 0.06$                      |
| 400                    | 400,000      | 0.015        | $\pm 0.03$                      |
| 1,000                  | 1,000,000    | 0.0095       | $\pm 0.02$                      |

Model B catches 10 of the 10 test frauds, model A catches 9. That is **one fraud** of difference — well inside either error bar. Compare two models on the **same** test cases and look at the cases where they disagree.

# Mini-Exercise

Recall is near 0.9, so  $SE = \sqrt{0.09/n}$  for  $n$  test frauds.

How many frauds does the test set need for  $SE = 0.03$  — and how many transactions is that?

# Mini-Exercise

Recall is near 0.9, so  $SE = \sqrt{0.09/n}$  for  $n$  test frauds.

How many frauds does the test set need for  $SE = 0.03$  — and how many transactions is that?

$0.09/n = 0.0009$ , so  $n = 100$  frauds — about **100,000 transactions** at one in 1,000. For  $SE = 0.015$  it would be four times as many.

# Takeaway

Every metric is an **estimate from a sample** and needs an **error bar**: a standard error, or a Beta posterior when the counts are small. For a rare class the error bar is set by the number of **rare cases**, not by the size of the test set.

# Module 3

---

## From Scores to Decisions

**Guiding Question:** The model says  $p(\text{fraud}) = 0.09$ . Check the transaction or let it through — and who decides where the line is?

# Two Mistakes, Two Prices

|                                          | transaction is fraud | transaction is legitimate |
|------------------------------------------|----------------------|---------------------------|
| <b>check it</b> (a text to the customer) | 0                    | €5 ( $c_{FP}$ )           |
| <b>let it through</b>                    | €500 ( $c_{FN}$ )    | 0                         |

The model says  $p = p(\text{fraud} \mid \mathbf{x})$ . Expected cost of checking:  $(1 - p) \cdot c_{FP}$ ; of letting it through:  $p \cdot c_{FN}$ .

At which  $p$  does checking become the cheaper action?

# Two Mistakes, Two Prices

|                                          | transaction is fraud | transaction is legitimate |
|------------------------------------------|----------------------|---------------------------|
| <b>check it</b> (a text to the customer) | 0                    | €5 ( $c_{FP}$ )           |
| <b>let it through</b>                    | €500 ( $c_{FN}$ )    | 0                         |

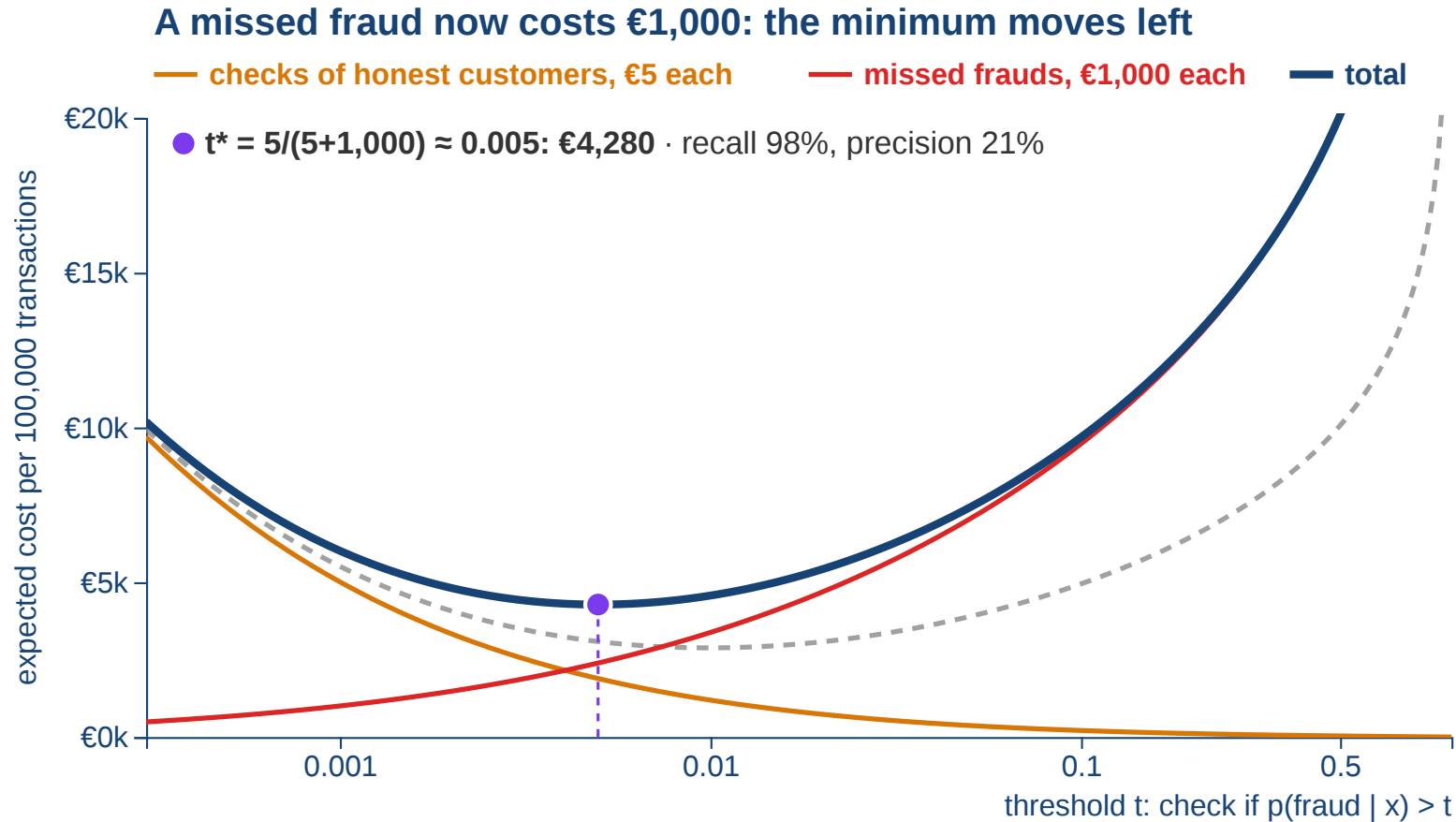
The model says  $p = p(\text{fraud} \mid \mathbf{x})$ . Expected cost of checking:  $(1 - p) \cdot c_{FP}$ ; of letting it through:  $p \cdot c_{FN}$ .

At which  $p$  does checking become the cheaper action?

$$p > p^* = \frac{c_{FP}}{c_{FP} + c_{FN}} = \frac{5}{505} \approx 0.01$$

Lecture 01's loan threshold, for fraud. Lecture 02's flagged transaction (9% fraud) is worth checking: 9% is nine times the break-even.

# Sweeping the Threshold



**Predict before the last click:** a missed fraud now costs €1,000. Does the best threshold move left or right — and does recall go up or down?

# Scoring the Probabilities Themselves

A threshold uses the score once. To evaluate the **probabilities**, score every prediction on held-out data:

## Log-loss

$$\text{log-loss} = -\frac{1}{n} \sum_{i=1}^n \log p_{\text{model}}(y_i \mid \mathbf{x}_i)$$

the held-out negative log-likelihood —  
Lecture 04's cross-entropy, on test data

## Brier score

$$\text{Brier} = \frac{1}{n} \sum_{i=1}^n (p_i - y_i)^2$$

squared error of the probability against the  
0/1 outcome

Both scores are **proper**: in expectation they are best when the model reports the true probability. Accuracy is not — it cannot tell 0.51 from 0.99.

# What the Scores Punish

| the model said $p = 0.09$ , and the transaction was... | log-loss term | Brier term |
|--------------------------------------------------------|---------------|------------|
| fraud                                                  | 2.41          | 0.83       |
| legitimate                                             | 0.09          | 0.008      |
| fraud, but the model had said $p = 0.001$              | <b>6.91</b>   | 0.998      |

Log-loss punishes **confident errors** without limit —  $p = 0$  on a fraud costs infinity; Brier is bounded by 1. Whether "0.09" comes true 9% of the time is Lecture 04's **reliability diagram**, now drawn on held-out data.

# Mini-Exercise

Checking costs €5, a missed fraud €500. The model says  $p = 0.02$ .

Check the transaction or let it through?

# Mini-Exercise

Checking costs €5, a missed fraud €500. The model says  $p = 0.02$ .

Check the transaction or let it through?

**Check.** Letting it through costs  $0.02 \cdot €500 = €10$  in expectation; checking costs  $0.98 \cdot €5 \approx €4.90$ . Equivalently,  $0.02 > p^* \approx 0.01$ .

# Takeaway

A **calibrated probability** plus the **costs** gives the decision: act when  $p > c_{FP} / (c_{FP} + c_{FN})$ . Precision and recall move along one curve as the threshold moves; **log-loss** and the **Brier score** evaluate the probabilities themselves.

# Module 4

---

## Keep the Loss: Empirical Risk Minimisation

**Guiding Question:** Every model you trained minimised something on the training data. What did we actually want — and what did the substitution throw away?

# What We Actually Want to Minimise

The **risk** of a hypothesis  $h$  under a loss  $L$  is its expected loss on the world:

$$R(h) = \mathbb{E}_{(\mathbf{x}, y) \sim p} [L(h(\mathbf{x}), y)]$$

## 0/1 loss

$R(h) = \text{err}^*(h)$  — the true error of Module 1

## Cost matrix

$R(h)$  = the expected cost of Module 3

It is the quantity that matters — and **uncomputable**:  $p$  is unknown.

# What Training Computes Instead

The **empirical risk** is the average loss on the training sample, and **empirical risk minimisation (ERM)** picks its minimiser:

$$\hat{R}(h) = \frac{1}{n} \sum_{i=1}^n L(h(\mathbf{x}_i), y_i), \quad \hat{h} = \arg \min_{h \in \mathcal{H}} \hat{R}(h)$$

| loss                                       | trained model                          | is $h(\mathbf{x})$ a probability?                                  |
|--------------------------------------------|----------------------------------------|--------------------------------------------------------------------|
| squared error                              | linear regression                      | the mean of a Gaussian — <b>yes</b> (Lecture 04)                   |
| cross-entropy $-\log p(y \mid \mathbf{x})$ | logistic regression, most neural nets  | <b>yes</b> — this is maximum likelihood for $p(y \mid \mathbf{x})$ |
| hinge $\max(0, 1 - y \cdot h(\mathbf{x}))$ | support vector machine                 | <b>no</b> — a distance to the margin                               |
| 0/1                                        | what we care about; no useful gradient | <b>no</b> — only a label                                           |

# Keep the Loss, Drop the Probabilities

## WHAT ERM WITH A HINGE OR 0/1 LOSS RETURNS

a score: 2.3

large means "more fraud-like" — nothing more

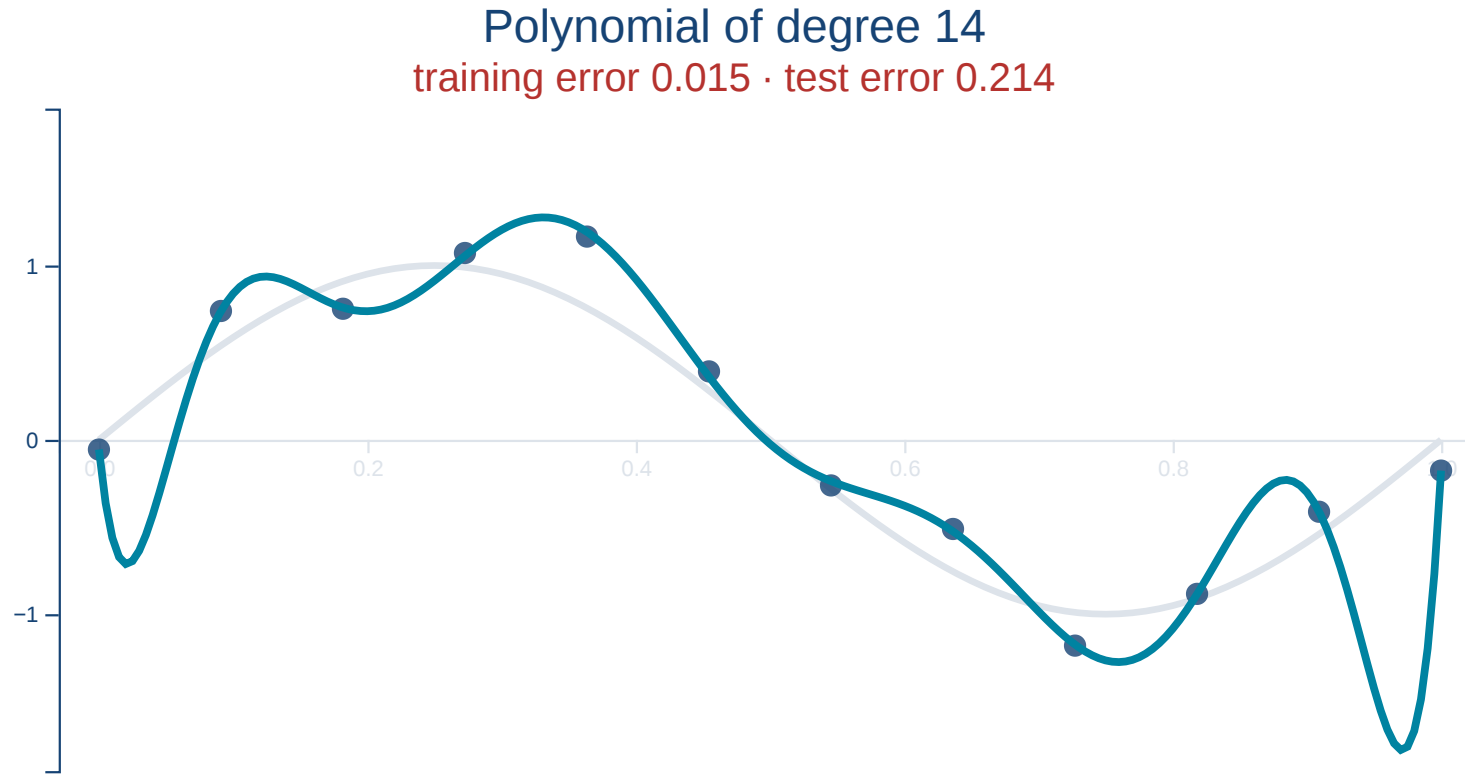
## WHAT IT CAN NO LONGER DO

- apply the cost threshold  $p > c_{FP} / (c_{FP} + c_{FN})$  of Module 3
- be scored by log-loss or Brier, or drawn in a reliability diagram
- put an error bar on its own confidence

To use it for a decision, **buy the probabilities back**: fit  $p = \sigma(a \cdot \text{score} + b)$  on held-out data (Platt scaling) — a logistic regression on the score, i.e. calibration.

Not every method is ERM: maximum likelihood for  $p(y \mid \mathbf{x})$  is ERM with loss  $-\log p$ ; generative maximum likelihood (naive Bayes) scores  $\log p(\mathbf{x}, y)$ , and Bayesian methods minimise nothing.

# The Generalisation Gap



$R(\hat{h}) = \hat{R}(\hat{h}) + (R(\hat{h}) - \hat{R}(\hat{h}))$ : the gap opens because  $\hat{h}$  was **chosen on the data that then measures it**, and it tends to grow with the capacity of  $\mathcal{H}$ .

# Structural Risk Minimisation — and the Prior Inside It

Minimise the empirical risk **plus** a complexity penalty:

$$\hat{h} = \arg \min_{h \in \mathcal{H}} \hat{R}(h) + \lambda \Omega(h)$$

With the loss  $-\log p(y \mid \mathbf{x}, \mathbf{w})$ , the MAP estimate of Lecture 04 has exactly this form:

$$\frac{1}{n} \sum_{i=1}^n -\log p(y_i \mid \mathbf{x}_i, \mathbf{w}) \quad \underbrace{-\frac{1}{n} \log p(\mathbf{w})}_{\lambda \Omega(\mathbf{w})}$$

The  $\frac{1}{n}$  is the point: the prior's weight **fades as data grows** — the pseudo-counts of Lecture 02's Beta prior. A Gaussian prior  $\mathcal{N}(0, \tau^2 \mathbf{I})$  gives  $\lambda \Omega(\mathbf{w}) = \|\mathbf{w}\|^2 / (2n\tau^2)$ .

# Mini-Exercise

A linear SVM gives a transaction the score **2.3**. The fraud team checks every transaction with  $p > 0.01$ .

Can they apply their rule to this transaction?

# Mini-Exercise

A linear SVM gives a transaction the score **2.3**. The fraud team checks every transaction with  $p > 0.01$ .

Can they apply their rule to this transaction?

**Not directly.** 2.3 is a distance to the margin, not a probability — the hinge loss dropped it. First calibrate on held-out data,  $p = \sigma(a \cdot 2.3 + b)$ , then compare that  $p$  with 0.01.

# Takeaway

Training minimises the **empirical risk** because the **risk** is out of reach. ERM keeps the **loss** and — unless the loss is a negative log-probability — drops the **probabilities**. Regularisers (penalties or priors) control the gap; error bars measure it; calibration buys the probabilities back.